

Slack bot – jak to funguje u MOBILMAJAK

Interní návod pro programátora · implementace do podobné aplikace

Tento dokument popisuje, jak je Slack bot napojený v aplikaci MOBILMAJAK (Django backend + React frontend) a jak stejný model přenést do podobné firemní aplikace. Nejde o oficiální Slack SDK tutorial – jde o naši provozní architekturu: fail-soft notifikace, propojení uživatelů přes e-mail a volitelný inbound (slash command + DM wizard).

1. Architektura – dvě vrstvy

Můžete nasadit jen vrstvu A. Vrstva B je navíc (zakládání úkolů ze Slacku).

Vrstva A – Outbound (DM notifikace)

Potřeba: Bot token xoxb-...

Slack API: `users.lookupByEmail`, `chat.postMessage`

Použití: termíny úkolů, SLA objednávek, denní report, recap ke směně, ...

Vrstva B – Inbound (slash / events / tlačítka)

Potřeba: Signing Secret + veřejné HTTPS URL na produkci

Slack API: Events (`message.im`), Slash Commands, Interactivity

Použití: /ukol wizard, text „úkol: ...“ v DM, tlačítka průvodce

Bez `SLACK_BOT_TOKEN` = notifikace se tiše přeskočí (no-op).

Bez `SLACK_SIGNING_SECRET` = inbound endpointy vrátí HTTP 503.

2. Nastavení Slack App (api.slack.com/apps)

- Create New App → From scratch → cílový workspace.

- So...

Jen pro vrstvu B nastavte Request URL (u nás produkce <https://mobilmajak.com>):

Slash command: <https://VASADOMENA/api/tasks/slack/commands/ukol/>

Interactivity: <https://VASADOMENA/api/tasks/slack/interactions/>

Event Subscriptions: <https://VASADOMENA/api/tasks/slack/events/>

+ Subscribe to bot events: `message.im`

3. Konfigurace na serveru

Env proměnné (nebo JSON secrets soubor; env má přednost):

`SLACK_BOT_TOKEN=xoxb-...`

`SLACK_SIGNING_SECRET=...` # jen pokud máte inbound

`MOBILMAJAK_APP_URL=https://mobilmajak.com` # odkazy v DM

volitelně fallback do kanálu:

`SLACK_TASKS_WEBHOOK_URL=https://hooks.slack.com/services/...`

Secrets držíme mimo git. Deploy nesmí přepsat produkční .env. Po změně secrets restartovat aplikaci (u nás unicorn).

4. Vrstva A – outbound DM (jádro)

Princip propojení uživatelů

Uživatel v aplikaci má e-mail. Stejný e-mail musí mít ve Slacku (firemní účet). Backend zavolá `users.lookupByEmail` → získá Slack user ID (U...). Pak `chat.postMessage` s `channel = U...` odešle přímou zprávu (DM). Manuální Slack ID v UI

není nutné; lookup se cacheuje v paměti procesu.

Fail-soft pravidla

- Chybí token / e-mail / Slack uživatel → skip + log, ne výjimka do HTTP requestu.

• Slack

Minimální klient (Python / requests)

```
def slack_api(token, method, payload):
    url = f"https://slack.com/api/{method}"
    headers = {"Authorization": f"Bearer {token}"}
    if method in ("users.lookupByEmail", "users.info"):
        r = requests.get(url, params=payload, headers=headers, timeout=15)
    else:
        headers["Content-Type"] = "application/json; charset=utf-8"
        r = requests.post(url, json=payload, headers=headers, timeout=15)
    r.raise_for_status()
    data = r.json()
    if not data.get("ok"):
        raise RuntimeError(data.get("error", "unknown"))
    return data
```

1) e-mail → Slack ID

```
data = slack_api(token, "users.lookupByEmail", {"email": email})
slack_id = data["user"]["id"]
```

2) DM

```
slack_api(token, "chat.postMessage", {
    "channel": slack_id,
    "text": text,
    "unfurl_links": False,
    "unfurl_media": False,
})
```

Kde notifikace volat

- Po business eventu (nový záznam, přiřazení, komentář, změna stavu) – sync nebo fronta.

• Per

5. Vrstva B – inbound (slash + DM + tlačítka)

Endpointy musí být CSRF-exempt, jen POST, a vždy nejdřív ověřit Slack podpis (HMAC-SHA256 přes Signing Secret, hlavičky X-Slack-Request-Timestamp a X-Slack-Signature, max. stáří 5 minut).

Chybí secret → 503 | Špatný podpis → 403 | url_verification → JSON {"challenge": "..."}

Events (message.im)

- Po url_verification zpracovat event_callback.

• Filtr

Slash command (u nás /ukol)

- Body je form-urlencoded (user_id, text, channel_id, ...).

• Odp

Interactivity

- Body: payload=<JSON> (block_actions / view_submission).

• Odp

6. Doporučená struktura modulů

```
slack_secrets.py # načtení token/secret (env > file)
slack_verify.py  # ověření HMAC podpisu
slack_client.py  # API wrapper: lookup, send_dm
slack_notify.py  # business notifikace (fail-soft)
slack_views.py   # HTTP endpoints (jen vrstva B)
slack_wizard.py  # stavový průvodce (jen vrstva B)
```

Doménové moduly (objednávky, reklamace, ...) volají jen tenkou notify_* funkci – nevolají Slack API přímo.

7. Jak to konkrétně běží u MOBILMAJAK

Referenční soubory v repu:

```
backend/tasks/slack_secrets.py  – secrets
backend/tasks/slack_verify.py   – podpis
backend/tasks/slack_notify.py   – klient + DM úkolů
backend/tasks/slack_views.py    – inbound HTTP
backend/tasks/slack_wizard.py   – /ukol průvodce
backend/orders/slack_notify.py   – příklad doménové notifikace
docs/slack-app-produkce.md      – checklist nastavení Slack App
docs/secrets-setup.md          – secrets na VPS
```

Příklady outbound událostí: přiřazení úkolu, due soon / overdue (cron), čeká schválení, dokončení, komentáře (dle preferencí uživatele), ranní recap ke směně, denní report prodejců, SLA připomínky objednávek.

Propojení Slack ↔ aplikace: e-mail v profilu = e-mail ve Slacku, účet aktivní. Test lookupu: curl na <https://slack.com/api/users.lookupByEmail?email=...>

8. Checklist pro novou aplikaci

Outbound only (minimum „jako my u notifikací“)

- Slack App + scopes + Install

• SLA

+ Inbound

- SLACK_SIGNING_SECRET + restart

• Veř

9. Časté chyby

Event URL „didn't respond with challenge“ → chybí Signing Secret (503) nebo špatná URL.

„Účet není propojený“ → jiný e-mail ve Slacku vs. v aplikaci.

Tlačítka nefungují → Interactivity URL / podpis.

Text v DM nefunguje → chybí event message.im.

Po změně scopes nic → Reinstall App (nový token).

Bot neodpovídá v DM → uživatel ještě neotevřel Messages tab appky.

Shrnutí: pro notifikace stačí bot token + e-mail lookup + chat.postMessage + fail-soft. Slash command a wizard jsou volitelná nadstavba. Dokument vychází z produkční implementace MOBILMAJAK.